

Algoritmos e Pensamento Computacional

Prof. Davi de Oliveira Cruz

Fio da aula: o sistema de notas cresce

O código já se organiza em funções e os dados em vetores. Hoje os dados ganham uma dimensão a mais — e depois um lugar permanente.

- **Matriz:** organiza notas de várias provas para vários alunos.
 - **Arquivo:** guarda tudo em disco, mesmo depois que o programa fecha.
 - Vetores e funções são a base dos dois.
 - Cada conceito resolve uma parte do mesmo problema.
-

Transição: do vetor à turma

Um vetor armazena as notas de uma prova. E se houver várias provas?

- Cada aluno pode ter várias notas.
 - Uma matriz organiza linhas (alunos) e colunas (provas).
 - Mesmo conceito de índice, agora com duas dimensões.
 - Depois, salvaremos tudo em arquivo.
-

Por que duas dimensões

Um vetor é uma linha de notas; a turma pede uma linha por aluno.

- Matriz é um vetor de vetores: `notas[3][4]` = 3 alunos × 4 provas.
 - O primeiro índice escolhe a linha (o aluno); o segundo, a coluna (a prova).
 - Mentalmente: uma tabela — `notas[i][j]` é a célula da linha `i`, coluna `j`.
-

Declaração de matriz

Matriz = array de arrays. Cada linha é um vetor.

Código

```
#include <stdio.h>
#define LINHAS 3
#define COLUNAS 4

int main(void) {
    int matriz[LINHAS][COLUNAS];

    float boletim[2][3] = {0};

    printf("boletim[0][0] = %.1f\n", boletim[0][0]);

    return 0;
}
```

O que observar

- Sintaxe: `tipo nome[L][C];`.
- Dimensões via `#define` são comuns.
- Memória contígua, organizada por linha.
- `{0}` zera todos os elementos.

Inicialização de matrizes

Chaves aninhadas preenchem linha por linha — as mesmas variantes do vetor valem aqui.

Código

```
#include <stdio.h>

int main(void) {
    int a[2][3] = {{1, 2, 3},
                  {4, 5, 6}}; /* completa */
    int b[2][3] = {0};        /* tudo zero */
    int c[2][3] = {{1}, {4}}; /* resto zero */
    int d[2][3];             /* lixo! */

    printf("c[0][1] = %d\n", c[0][1]);
    printf("c[1][0] = %d\n", c[1][0]);

    return 0;
}
```

O que observar

- Inicialização parcial completa o resto com `0`.
- `{0}` é o jeito curto de zerar a grade inteira.
- Sem inicializar, `d` guarda valores indeterminados — lixo.
- Igual ao vetor: matriz nova, inicialize.

Indexação dupla

`matriz[i][j]` acessa a linha `i` e a coluna `j`.

Código

```
#include <stdio.h>

int main(void) {
    int notas[2][3] = {
        {8, 7, 9},
        {6, 8, 7}
    };

    notas[1][2] = 10;

    printf("%d\n", notas[0][1]);

    return 0;
}
```

O que observar

- Linhas e colunas começam em 0.
- Inicialização literal usa chaves aninhadas.
- Atribuição direta: `notas[i][j] = valor;`
- Ordem dos índices importa: `linha` depois `coluna`.

Ler uma matriz com `scanf`

O laço duplo preenche a grade linha por linha — fixa `i`, varre `j`.

Código

```
#include <stdio.h>
#define LINHAS 3
#define COLUNAS 4

int main(void) {
    float notas[LINHAS][COLUNAS];

    for (int i = 0; i < LINHAS; i++) {
        for (int j = 0; j < COLUNAS; j++) {
            printf("nota[%d][%d]: ", i, j);
            scanf("%f", &notas[i][j]);
        }
    }

    return 0;
}
```

O que observar

- `i` fixo (aluno), `j` varre as provas.
- `scanf` precisa do `&` antes de `notas[i][j]`.
- A ordem dos laços define a ordem das perguntas.
- Total de leituras: `LINHAS * COLUNAS`.

Percorrer uma matriz

Laço externo percorre linhas; interno, colunas.

Código

```
#include <stdio.h>
#define LINHAS 3
#define COLUNAS 4

int main(void) {
    int matriz[LINHAS][COLUNAS];
    int i, j, valor = 1;

    for (i = 0; i < LINHAS; i++) {
        for (j = 0; j < COLUNAS; j++) {
            matriz[i][j] = valor;
            valor++;
        }
    }

    for (i = 0; i < LINHAS; i++) {
        for (j = 0; j < COLUNAS; j++) {
            printf("%4d", matriz[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

O que observar

- Total de acessos: `LINHAS * COLUNAS`.
- `printf("\n")` após o laço interno imprime por linha.
- Invertida a ordem dos laços, a saída troca.

Rastreando laços aninhados

O laço interno completa uma linha inteira antes de o externo avançar.

Iteração	i	j	Posição gravada
1	0	0	[0][0]
2	0	1	[0][1]
3	0	2	[0][2]
4	0	3	[0][3]
5	1	0	[1][0]
...			
12	2	3	[2][3]

Soma por linha: média do aluno

Fixa a linha (o aluno) e varre as colunas (as provas).

Código

```
#include <stdio.h>
#define ALUNOS 3
#define PROVAS 4

int main(void) {
    float notas[ALUNOS][PROVAS] = {
        {8.0f, 7.0f, 9.0f, 6.0f},
        {5.0f, 6.0f, 7.0f, 8.0f},
        {9.0f, 8.0f, 7.0f, 6.0f}
    };

    for (int i = 0; i < ALUNOS; i++) {
        float soma = 0;
        for (int j = 0; j < PROVAS; j++) {
            soma += notas[i][j];
        }
        printf("aluno %d: media %.1f\n",
            i, soma / PROVAS);
    }
    return 0;
}
```

O que observar

- `soma` zera a cada linha — dentro do laço externo.
- Responde: "qual a média de cada aluno?"
- Mesmo esqueleto serve para maior nota do aluno, etc.

Soma por coluna: média da prova

Inverte os laços — fixa a coluna (a prova) e varre os alunos.

Código

```
#include <stdio.h>
#define ALUNOS 3
#define PROVAS 4

int main(void) {
    float notas[ALUNOS][PROVAS] = {
        {8.0f, 7.0f, 9.0f, 6.0f},
        {5.0f, 6.0f, 7.0f, 8.0f},
        {9.0f, 8.0f, 7.0f, 6.0f}
    };

    for (int j = 0; j < PROVAS; j++) {
        float soma = 0;
        for (int i = 0; i < ALUNOS; i++) {
            soma += notas[i][j];
        }
        printf("prova %d: media %.1f\n",
            j, soma / ALUNOS);
    }
    return 0;
}
```

O que observar

- Laço externo em **j**, interno em **i** — a ordem troca.
- Responde: "qual a média de cada prova?"
- Trocar os laços = trocar a pergunta respondida.

Maior e menor em matriz

O mesmo padrão do vetor, agora com laço duplo.

Código

```
#include <stdio.h>
#define ALUNOS 3
#define PROVAS 4

int main(void) {
    float notas[ALUNOS][PROVAS] = {
        {8.0f, 7.0f, 9.0f, 6.0f},
        {5.0f, 6.0f, 7.0f, 8.0f},
        {9.0f, 8.0f, 7.0f, 6.0f}
    };
    float maior = notas[0][0];
    float menor = notas[0][0];

    for (int i = 0; i < ALUNOS; i++)
        for (int j = 0; j < PROVAS; j++) {
            if (notas[i][j] > maior)
                maior = notas[i][j];
            if (notas[i][j] < menor)
                menor = notas[i][j];
        }

    printf("maior = %.1f, menor = %.1f\n",
           maior, menor);
    return 0;
}
```

O que observar

- `maior` / `menor` iniciam em `notas[0][0]` — nunca em 0.
- Os dois `if` convivem no mesmo laço interno.
- Responde: "qual a melhor e a pior nota da turma?"

Busca em matriz

Procurar uma nota devolve um par: a posição `[i][j]`.

Código

```
#include <stdio.h>
#define ALUNOS 3
#define PROVAS 4

int main(void) {
    float notas[ALUNOS][PROVAS] = {
        {8.0f, 7.0f, 9.0f, 6.0f},
        {5.0f, 6.0f, 7.0f, 8.0f},
        {9.0f, 8.0f, 7.0f, 6.0f}
    };
    float procurada;
    int linha = -1, coluna = -1;

    scanf("%f", &procurada);

    for (int i = 0; i < ALUNOS; i++)
        for (int j = 0; j < PROVAS; j++)
            if (notas[i][j] == procurada) {
                linha = i;
                coluna = j;
            }

    printf("posicao = [%d][%d]\n",
           linha, coluna);
    return 0;
}
```

O que observar

- `linha = -1, coluna = -1` significa "não encontrada".
- Resposta é um par de índices, não um índice só.
- `break` sairia só do laço interno — aqui varremos tudo.
- Mesmo `==` exato de float: cuidado com valores calculados.

Operações comuns sobre matrizes

Soma de matrizes

- $\text{resultado}[i][j] = a[i][j] + b[i][j]$
- Exige mesmas dimensões em A e B

Soma da diagonal principal

- Soma apenas $m[i][i]$, em um único laço
- Só faz sentido em matriz quadrada

Transposta simples

- $t[j][i] = m[i][j]$
- Dimensões invertidas: COLUNAS x LINHAS

Código: soma e diagonal

Código

```
#include <stdio.h>
#define L 3

int main(void) {
    int a[L][L] = {{1, 2, 3},
                  {4, 5, 6},
                  {7, 8, 9}};
    int b[L][L] = {{1, 1, 1},
                  {1, 1, 1},
                  {1, 1, 1}};

    int r[L][L];
    int diagonal = 0;

    for (int i = 0; i < L; i++)
        for (int j = 0; j < L; j++)
            r[i][j] = a[i][j] + b[i][j];

    for (int i = 0; i < L; i++)
        diagonal += a[i][i];

    printf("r[1][1] = %d\n", r[1][1]);
    printf("diagonal = %d\n", diagonal);
    return 0;
}
```

O que observar

- A soma percorre posição a posição: `r[i][j]`
`= a[i][j] + b[i][j]`.
- A diagonal acessa só `a[i][i]` — exige matriz quadrada.
- Matrizes devem ter as mesmas dimensões.
- `r[1][1]` imprime `6`; `diagonal` imprime `15`.

Código: transposta

Código

```
#include <stdio.h>
#define L 2
#define C 3

int main(void) {
    int m[L][C] = {{1, 2, 3}, {4, 5, 6}};
    int t[C][L];

    for (int i = 0; i < L; i++) {
        for (int j = 0; j < C; j++) {
            t[j][i] = m[i][j];
        }
    }

    printf("t[0][1] = %d\n", t[0][1]);

    return 0;
}
```

O que observar

- Elemento `m[i][j]` passa a ocupar `t[j][i]`.
- Matriz destino tem dimensões trocadas.
- Não altera a matriz original.
- `t[0][1]` recebe `m[1][0]` — imprime `4`.

Cuidados em matrizes 2D

- Trocar a ordem dos índices: `matriz[linha][coluna]` x `matriz[coluna][linha]`.
- Percorrer a matriz com um só `for` sem saber mapear o índice em linha e coluna.
- Esquecer que os índices vão até `LINHAS-1` e `COLUNAS-1`.
- Somar duas matrizes com dimensões diferentes.
- Calcular a diagonal percorrendo a matriz inteira em vez de só `m[i][i]`.
- Esquecer de trocar as dimensões da matriz destino na transposta.
- Confundir transposta com alteração da própria matriz original.

Exercício 8.1 — Matriz de notas

Notas de várias provas organizadas em grade: linhas = alunos, colunas = provas.

- Ler duas matrizes 3×3 de inteiros, A e B, com laço duplo (`i` linha, `j` coluna).
- Calcular e exibir a matriz soma de A e B.
- Calcular e exibir a soma da diagonal principal de A (posições `i == j`).
- Calcular e exibir a transposta de A — `t[j][i] = a[i][j]`, atenção às dimensões.

Transição: da memória para o disco

As notas estão na memória enquanto o programa roda. Como preservá-las?

- Variáveis e arrays desaparecem ao encerrar.
 - Arquivo texto grava dados em disco.
 - Programa pode reabrir o arquivo depois.
 - Cada registro vira uma linha no arquivo.
-

Conceito de arquivo texto e `FILE *`

Arquivo texto: sequência de caracteres organizada em linhas, persistente em disco.

- `FILE *` é um ponteiro opaco para o arquivo.
 - Todo uso depende de `#include <stdio.h>`.
 - `stdin`, `stdout`, `stderr` já são `FILE *` do sistema.
 - O ponteiro representa um cursor que avança na leitura/escrita.
-

Exemplo: declarando `FILE *`

`FILE *` representa o arquivo usado por funções de `stdio.h`.

Código

```
#include <stdio.h>

int main(void) {
    FILE *arquivo = NULL;

    if (arquivo == NULL) {
        printf("Arquivo ainda nao foi aberto.\n");
    }

    return 0;
}
```

O que observar

- `FILE *` armazena referência para fluxo.
- `NULL` indica que nenhum arquivo foi associado.
- `fopen` atribui um fluxo quando abertura funciona.
- Testar `NULL` evita usar ponteiro inválido.

Modos **r**, **w** e **a**

Modo	Nome	Arquivo precisa existir?	Efeito
"r"	Leitura	Sim — falha (<code>NULL</code>) se não existir	Lê do início, não apaga
"w"	Escrita	Não — cria se faltar	Apaga conteúdo anterior
"a"	Acréscimo	Não — cria se faltar	Escreve ao final, sem apagar

Escrita com `fprintf`

`fprintf` grava no arquivo como `printf` grava na tela.

- `fprintf(arquivo, "formato", ...)` envia dados para o `FILE *`.
 - Usa os mesmos especificadores de `printf`.
 - Boa prática: terminar cada registro com `\n`.
 - A escrita pode ficar em buffer até `fclose`.
 - Arquivo precisa estar aberto em modo `"w"` ou `"a"`.
-

Exemplo: gravando um registro

`fprintf` envia saída formatada para o arquivo.

Código

```
#include <stdio.h>

int main(void) {
    FILE *arquivo =
        fopen("notas.txt", "w");

    if (arquivo != NULL) {
        int matricula = 1023;
        char nome[] = "Ana";
        float nota = 8.5f;

        fprintf(arquivo,
            "%d %s %.1f\n",
            matricula, nome, nota);
        fclose(arquivo);
    }

    return 0;
}
```

O que observar

- Modo `"w"` cria ou substitui conteúdo.
- Primeiro argumento é `FILE *`.
- `\n` separa registros em linhas.
- `fclose` encerra o fluxo.

```
1023 Ana 8.5
```

Leitura: `fscanf` X `fgets`

`fscanf`

- Formato fixo, como `scanf`
- Converte valores direto para variáveis
- Retorna quantidade de conversões
- EOF indica fim do arquivo

`fgets`

- Lê uma linha inteira
- Guarda em vetor de `char`
- Mais seguro contra estouro
- Retorna `NULL` no fim do arquivo

Exemplo: lendo com `fscanf`

Testar quantidade de campos lidos evita registro incompleto.

Código

```
#include <stdio.h>

int main(void) {
    FILE *arquivo =
        fopen("notas.txt", "r");
    int matricula;
    char nome[50];
    float nota;

    if (arquivo != NULL) {
        while (fscanf(arquivo,
            "%d %49s %f",
            &matricula, nome,
            &nota) == 3) {
            printf("%d | %s | ",
                matricula, nome);
            printf("%.1f\n", nota);
        }
        fclose(arquivo);
    }

    return 0;
}
```

O que observar

- `== 3` exige três campos válidos.
- `%49s` limita nome sem espaços.
- `fscanf` retorna número de conversões.
- `fclose` deve ocorrer após leitura.

Exemplo: lendo com `fgets`

`fgets` lê linha inteira respeitando tamanho do buffer.

Código

```
#include <stdio.h>

int main(void) {
    FILE *arquivo =
        fopen("notas.txt", "r");
    char linha[100];

    if (arquivo != NULL) {
        while (fgets(linha,
                     sizeof(linha),
                     arquivo) != NULL) {
            printf("Linha: %s", linha);
        }
        fclose(arquivo);
    }

    return 0;
}
```

O que observar

- `sizeof(linha)` limita caracteres lidos.
- Retorno `NULL` indica fim do arquivo.
- Linha normalmente contém `\n`.
- Remover `\n` quando comparar textos.

fgets e o caractere `\n`

`fgets` mantém a quebra de linha no vetor.

Código

```
#include <stdio.h>

int main(void) {
    char nome[50];

    printf("Digite um nome: ");
    fgets(nome, sizeof(nome), stdin);

    for (int i = 0; nome[i] != '\0'; i++) {
        if (nome[i] == '\n') {
            nome[i] = '\0';
            break;
        }
    }

    printf("nome = %s\n", nome);

    return 0;
}
```

O que observar

- `sizeof(nome)` limita a leitura.
- `\0` marca o fim da string.
- Substituir `\n` por `\0` limpa o texto.
- Excesso de entrada permanece no fluxo.

Verificação de erros de abertura

`fopen` retorna `NULL` em caso de falha.

- Sempre verificar `if (arquivo == NULL)` logo após `fopen`.
 - Usar ponteiro `NULL` como arquivo válido causa comportamento indefinido.
 - Boa prática: exibir mensagem de erro clara.
 - Verificação é essencial em modo `"r"`.
-

Exemplo: tratando falha de abertura

Programa deve testar retorno antes de usar o fluxo.

Código

```
#include <stdio.h>

int main(void) {
    FILE *arquivo =
        fopen("falta.txt", "r");

    if (arquivo == NULL) {
        printf("Erro: nao foi possivel abrir arquivo.\n");
        return 1;
    }

    fclose(arquivo);

    return 0;
}
```

O que observar

- Modo `"r"` exige arquivo existente.
- `NULL` indica falha de abertura.
- Retorno antecipado evita operar em ponteiro inválido.
- Arquivo aberto com sucesso também precisa de `fclose`.

Do vetor ao arquivo e de volta

O ciclo completo do sistema de notas: gravar, fechar, reabrir, reler.

Código

```
#include <stdio.h>
#define TAMANHO 5

int main(void) {
    float notas[TAMANHO] = {8, 7, 9, 6, 8};
    FILE *arq;

    arq = fopen("notas.txt", "w");
    if (arq == NULL) return 1;
    for (int i = 0; i < TAMANHO; i++) {
        fprintf(arq, "%.1f\n", notas[i]);
    }
    fclose(arq);

    arq = fopen("notas.txt", "r");
    if (arq == NULL) return 1;
    for (int i = 0; i < TAMANHO; i++) {
        fscanf(arq, "%f", &notas[i]);
    }
    fclose(arq);

    printf("notas[2] = %.1f\n", notas[2]);
    return 0;
}
```

O que observar

- O vetor vai para o disco e volta.
- "w" grava; "r" relê — `fclose` entre os dois.
- `NULL` verificado após cada `fopen`.

```
8.0
7.0
9.0
6.0
8.0
```

Cuidados em arquivos

- Esquecer de chamar `fclose`, deixando dados no buffer.
- Não verificar o retorno de `fopen`.
- Confundir modos `"w"` e `"a"`, apagando dados.
- Usar `fscanf` com `%s` para nomes com espaço, sem perceber que pára em espaço.
- Usar `fgets` sem remover `\n` ao comparar ou converter.
- Abrir em `"r"` algo que ainda não foi gravado.
- Não usar `sizeof(buffer)` como segundo argumento de `fgets`.

Exercício 8.2 — Arquivo de notas

Persistir as notas da turma em arquivo texto — o programa pode ser fechado e os dados continuam.

- Ler um vetor de 5 notas do teclado.
 - Gravar em `notas.txt`, uma nota por linha, com `fprintf`.
 - Reabrir em modo `"r"` e verificar `if (arquivo == NULL)` logo após cada `fopen`.
 - Reler com `fscanf`, exibir as notas e fechar com `fclose`.
-

Lista de Presença



Escaneie o QR Code para registrar presença.

Referências

- DEITEL, P. J.; DEITEL, H. M. **C: como programar**. 6. ed. São Paulo: Pearson, 2011.
 - FORBELLONE, A. L. V. **Lógica de Programação: a construção de algoritmos e estruturas de dados**. 3. ed. São Paulo: Pearson Prentice Hall, 2005.
 - SANTOS, M. G. **Algoritmos e programação**. Porto Alegre: SAGAH, 2018.
 - ASCENCIO, A. F. G.; CAMPOS, E. A. V. **Fundamentos da programação de computadores**. 3. ed. São Paulo: Pearson, 2012.
 - MIZRAHI, V. V. **Treinamento em linguagem C**. 2. ed. São Paulo: Pearson, 2008.
-